Build a simple e-commerce website

Web Development Internship Assessment | CubeTech Innovations

1. Project objective

Create a responsive e-commerce website for a **fictional store** with two connected interfaces: a customer website and an admin dashboard. Demonstrate a complete journey from browsing products to placing an order and managing it in the admin dashboard.

Completion window: Five working days from the start date provided by CubeTech. Plan approximately 15-20 hours in total, depending on your experience and chosen stack. Keep the implementation simple. If the work exceeds your available time, communicate early and document what remains.

2. Required scope at a glance

Customer website	Admin dashboard
Home, product listing, product details, cart, checkout, and confirmation.	Demo login, overview, product/category management, orders, order details, and a customer summary.
Search, category filter, price sorting, quantity controls, and persistent cart.	Add/edit/delete products and categories; view orders and update their status.

3. Sample store and data

- Choose a store name and a simple visual identity. Seed **at least 10 products across 3 categories**. Use fictional customer details and appropriate sample images.
- Use **PHP**, with prices and totals displayed to two decimal places. Include at least one inactive product and one active product with zero stock to demonstrate their behavior.
- Sample payment methods: Cash on Delivery, E-Wallet, and Bank Transfer. These are selections only; no payment collection, QR payment, or real gateway is required.

4. What matters most

Prioritize a working customer-to-admin flow, correct calculations, clear validation, readable code, and usable mobile screens. You may use a familiar framework, UI library, and storage approach. Extra features are optional and do not replace the required flow.

Out of scope: Production payment processing, shipping integrations, refunds, tax rules, multi-vendor features, and production security hardening. A separate native mobile application is not required.

For progress updates, briefly report: completed work, next step, blockers, and approximate time spent. Communicate blockers early through the assessment email thread.

Customer website

Build a clear shopping experience that works on phones and desktop browsers.

5. Home page

Include a store name/logo, navigation menu, promotional banner, category links, featured products, and footer with sample contact information and relevant social links. Navigation must lead to working pages or sections. Label fictional contact details clearly.

6. Product listing

- Show an image, name, price, category, stock availability, View Details action, and Add to Cart action for each visible product.
- Support case-insensitive product-name search, category filtering, and price sorting from low to high and high to low. Search, filtering, and sorting should work together.
- Use a responsive product grid. Show a clear empty state when no products match.
- **Inactive products:** Hide them from customer listings and prevent ordering through a direct product link. **Active products with zero stock:** Keep them visible with an Out of Stock label and disable Add to Cart.

7. Product details

Show the product image, name, price, description, available stock, quantity selector, and Add to Cart action. Show up to three related active products from the same category, excluding the current product. If none exist, omit the related-products section.

8. Shopping cart

- Show product names, unit prices, quantities, line totals, subtotal, and total. Customers can update quantities and remove items.
- Adding the same product again must increase its quantity rather than create duplicate lines. Quantity must be a positive whole number and must not exceed available stock.
- Keep cart contents after refreshing the browser. Prevent checkout with an empty cart.
- For this assessment, **subtotal equals total**: no shipping fee, tax, voucher, or service charge is required. Use safe money calculations to avoid rounding errors.

Example: Two units at PHP 120.00 plus one unit at PHP 85.50 must produce a total of PHP 325.50.

Checkout & order rules

An order is successful only after it has been saved.

9. Checkout form

Field	Validation
Customer name	Required; reject blank or whitespace-only input.
Email address	Required; validate email format.
Contact number	Required; allow common phone formatting without accepting an empty value.
Delivery address	Required; a single address field is sufficient.
Payment method	Required; choose from the three sample methods.
Order notes	Optional.

- Display the items, quantities, current unit prices, line totals, and final total before submission.
- Recheck product availability, price, and stock against the shared data source at submission. If a price or availability changed, show the change and require the customer to review before trying again.
- Disable the submit button while saving. On failure, show a useful error and keep the cart. On success, show a unique order number, customer details, payment method, item summary, total, and initial **Pending** status.
- Save the order and its items together, then clear the cart. Confirmation must not appear if saving failed.

10. Stock and order behavior

- Reduce stock when an order is successfully placed. Do not allow a stock quantity below zero.
- Save a snapshot of item name and unit price in each order line. Later product changes must not alter an existing order.
- Use the sequence **Pending** → **Confirmed** → **Preparing** → **Shipped** → **Completed**. Permit cancellation from Pending or Confirmed only. Completed and Cancelled are final statuses.
- When cancelling an eligible order, restore its stock exactly once. Reject repeated cancellation or backward status changes.
- Do not require a customer account. Status updates on a customer tracking page are optional; the admin order details must always show the saved status.

Backend implementations: Validate on the server and calculate totals from stored prices. Client-side validation alone is insufficient. **LocalStorage implementations:** Apply the same rules in shared application logic, and disclose that browser storage is user-editable and not production-safe.

Admin dashboard

Keep admin screens functional and consistent with customer-facing data.

11. Demo login and overview

Provide a simple admin login/logout flow and prevent direct access to admin routes without a demo session. Simulated authentication is acceptable. Include demo-only credentials in the README; never include real credentials. Clearly label simulated login as unsuitable for production.

Summary card	Definition
Total Products	All product records, including inactive products.
Total Orders	All saved orders, including cancelled orders.
Pending / Completed Orders	Counts of orders with the respective status.
Total Customers	Unique customers derived from normalized checkout emails.
Total Sales	Total value of Completed orders only; exclude all other statuses.

12. Product management

- List, search, add, edit, and delete products. Filter by category and displayed status.
- Fields: name, image URL or bundled image reference, category, description, price, stock quantity, and Active/Inactive setting. File upload is optional.
- Validate required fields, a price greater than zero, and a whole-number stock quantity of zero or more. Stock availability is derived: an active product with zero stock displays Out of Stock.
- Confirm deletion. Block hard deletion of a product referenced by an order and offer to mark it inactive instead. Preserve order history.

13. Category and customer management

Categories: View, add, edit, and delete categories. Require a non-blank name and prevent duplicate names ignoring case. Block deletion while any product is assigned to the category; explain why.

Customers: A read-only list derived from orders is sufficient. Show name, normalized email, contact number, order count, and total value of Completed orders. Use the latest submitted name/contact for repeat emails. Customer registration and account-status management are not required because checkout is guest-based.

Orders & shared data

The customer website and admin dashboard must use the same underlying data.

14. Order management

- List orders newest first with order number, customer name, date/time, total, payment method, status, and a View Details action.
- Order details must show customer/contact information, delivery address, products, quantities, unit prices, line totals, subtotal, total, payment method, status, and notes.
- Allow only the status transitions defined on page 3. Display feedback after a successful update and an error when saving fails.
- Provide a useful empty state. Manual refresh is sufficient; automatic real-time updates are optional.

15. Choose and document one storage approach

Approach	Assessment expectations
Browser demo: LocalStorage	Allowed for frontend-focused work. Store shared products, categories, orders, and cart persistently. Use one site origin for customer and admin routes. Document that it works only in the same browser/profile and is not a backend or multi-device solution. Static JSON alone is not sufficient for saving changes.
Backend/database or managed service	Allowed for full-stack work. Use a real shared data store, with server-side validation and authoritative price/stock calculation. Provide API/schema and environment setup instructions. For a hosted admin area, enforce real authentication and authorization.

16. Verify the connected flow

- A product added by admin appears on the customer site after refresh. An inactive product disappears. Price, description, and stock changes become visible after refresh.
- A submitted customer order appears in admin with matching items and total. Stock and dashboard figures update consistently.
- Data survives refresh; backend data also survives application restarts. Existing order snapshots remain unchanged after product edits.

Suggested data: Categories; Products; Orders; Order Items. A separate Customer table is optional when the customer list is derived from orders. For full-stack work, describe relationships and provide database migrations or seed instructions.

Plan & acceptance checks

Use five working days and keep a short record of what you tested.

Day	Suggested milestone
1	Plan screens/data, choose the stack/storage, create the repository, and seed products/categories.
2	Build home, listing, details, search/filter/sort, and cart.
3	Implement checkout, validation, saved orders, and stock updates.
4	Build demo login, admin CRUD, orders, customer summary, and dashboard figures.
5	Test the connected flow, fix issues, check mobile layouts, document, and record a demo.

17. Test log

Submit a Pass / Fail / Not tested result and a short note for each check. Manual testing is acceptable; automated tests are optional.

ID	Acceptance check
01	Search, category filter, and both price sorts work together. No-match and invalid product states are handled.
02	Cart merges duplicate products, persists after refresh, and calculates the PHP 325.50 example correctly.
03	Empty checkout, invalid customer fields, and invalid/excessive quantities are blocked.
04	Inactive/out-of-stock items cannot be ordered. Checkout detects price or availability changes before saving.
05	A successful order shows confirmation, appears in admin, reduces stock, and clears the cart. Failed saves preserve the cart.
06	Admin product/category changes appear on the customer site after refresh. Referenced records cannot be deleted.
07	Valid statuses save; invalid transitions fail. Eligible cancellation restores stock once, including after refresh.
08	Completed-order totals and unique-email customer counts match the underlying orders.
09	Existing order prices/names remain unchanged after product edits. Data persists as described for the chosen approach.
10	Customer/admin screens work at approximately 375 px and 1280 px widths. No clipped controls, horizontal page scrolling, or major console errors.

Submission & evaluation

Make your project easy to open, run, review, and discuss.

18. Required submission

- **Repository:** A GitHub link accessible to the reviewer. Private repositories are acceptable; request the reviewer account for access.
- **README:** Overview, technologies, chosen storage approach and limitations, install/run steps, environment placeholders, seed/reset instructions, customer/admin routes, and demo credentials.
- **Evidence:** Desktop/mobile screenshots, the completed test log, and a 3-5 minute demo video showing a purchase and its management in admin.
- **Explanation:** A brief system-flow/data-model summary, known issues, unfinished requirements, and a disclosure of AI tools, templates, and tutorials used.

Live links: Include customer and admin links when safely hosted. A local working demo plus video is acceptable if hosting is unavailable. A browser-only demo may be hosted with fictional data and a clear demo label; simulated login provides no security. A public dashboard backed by a shared database must use real access controls. Do not expose secrets or real personal data.

Send to: admin@sales-cubetech.cloud
Subject: Web Development Intern Assessment - [Full Name]
Include the repository, demo/video links, and access instructions. Follow the specific deadline in your assessment email.

19. Evaluation criteria

Criterion	Weight
Customer website functionality	20%
Admin dashboard functionality	25%
UI and user experience	15%
Code quality and organization	15%
Mobile responsiveness	10%
Data integration and system flow	10%
Documentation	5%

Optional extras: Image upload, wishlist, customer accounts, order tracking, charts, CSV export, pagination, or dark mode. Complete the required flow first. No extra feature is necessary for a strong submission.

AI and review: AI assistance is allowed when disclosed. You must understand, test, and explain the submitted code. Be ready to trace an order through the system and make one small requested change. We assess reasoning and learning potential as well as the finished interface.